

Université Paris Cité — UFR de Mathématiques et Informatique

Licence 2 Informatique — Projet L2G1

Plan de Test

Outil de traçabilité automatisé des exigences

Référent / Maîtrise d'ouvrage : M. Jérémie Meynadier

Équipe : Emilie Huang | Lamia Arrahmane | Amel Kheloul | Yseult Hellé Riglet

Version A01 — Avril 2026

1. Introduction

Le présent document constitue le Plan de Test du projet L2G1, réalisé dans le cadre d'une Licence 2 d'Informatique à l'Université Paris Cité. Il définit l'ensemble des tests fonctionnels, d'intégration et unitaires permettant de vérifier que l'outil de traçabilité automatisé développé est conforme aux exigences spécifiées dans le Cahier des Charges.

1.1. Objectifs et méthodes

L'objectif de ce document est de décrire de façon précise tous les tests à effectuer sur le logiciel livré. Chaque test est lié à une ou plusieurs exigences du Cahier des Charges afin d'assurer une couverture complète des fonctionnalités (EX-01 à EX-22).

La méthode employée est une validation fonctionnelle boîte noire : les tests vérifient le comportement observable de l'outil (entrées/sorties) sans examiner le code source. Pour chaque test : saisie de données en entrée définies → exécution → observation du résultat → comparaison avec le résultat attendu → verdict (PASS / FAIL / N/A).

Deux types d'objectifs sont couverts :

- Cas nominaux (N) : le chemin normal où tout se passe bien.
- Cas de robustesse (R) : les situations dégradées, en dehors du périmètre normal, où le programme ne doit pas planter.

1.2. Documents de référence

- Cahier des Charges du Projet L2G1 (version A02, Mai 2026)
- Cahier de Recettes du Projet L2G1 (version A02, Mai 2026)
- Cahier de Conception Générale (version A02, Mai 2026)
- Cahier de Conception Détaillée (version A02, Mai 2026)

2. Guide de lecture

Ce document est destiné à deux types de lecteurs : l'équipe de développement (maîtrise d'œuvre) et le référent pédagogique (maîtrise d'ouvrage).

2.1. Maîtrise d'œuvre

La maîtrise d'œuvre désigne l'ensemble des acteurs en charge de la réalisation concrète du projet. Elle assure la conception, le développement et la livraison du produit final dans le respect des exigences définies par le client.

L'équipe de développement utilise ce document pour comprendre les critères de validation attendus et préparer les démonstrations. Elle doit s'assurer que chaque test peut être exécuté avec succès avant la recette finale.

Membres de l'équipe :

- Emilie Huang
- Lamia Arrahmane
- Amel Kheloul
- Yseult Hellé Riglet

Encadrant : M. Jérémy Meynadier, responsable de la maîtrise d'œuvre. Il accompagne l'équipe dans ses choix techniques et méthodologiques et valide la conformité des livrables.

2.2. Maîtrise d'ouvrage

La maîtrise d'ouvrage désigne le client du projet. Elle est responsable de l'expression des besoins, de la définition des objectifs attendus et de la validation du produit final. Elle s'assure que la solution livrée répond aux exigences formulées dans le Cahier des Charges.

Responsable maîtrise d'ouvrage : M. Jérémy Meynadier. Il prend les décisions finales concernant la recevabilité du produit et valide la conformité des livrables avec les besoins exprimés.

2.3. Nomenclature des tests

Chaque test suit la nomenclature : OBJ_[N|R]_[MODULE]_[EX-XX]_[Lettre]

- N = Nominal, R = Robustesse
- IMP = Import, PAR = Parsing, TRC = Traçabilité, GUI = Interface, INT = Intégration, UT = Unitaire
- EX-XX = exigence du CdC couverte en priorité
- Lettre = A, B, C... pour distinguer plusieurs tests couvrant la même exigence

2.4. Verdicts

Chaque test reçoit un verdict selon le tableau ci-dessous. Le verdict apparaît dans la colonne "Résultat" de chaque tableau de pas de tests.

Verdict	Signification
PASS	Le comportement observé correspond exactement au résultat attendu.
FAIL	Le comportement observé ne correspond pas au résultat attendu.
N/A	Non applicable — le test ne peut pas être réalisé dans le contexte du programme.

3. Concepts de base

Les principes fondamentaux sur lesquels repose ce projet sont les suivants :

- Exigence : chaque exigence possède un identifiant unique, un contenu et peut contenir des liens de traçabilité.
- Identifiant : clé technique unique (ex : EXIGENCE_CLIENT_VOIE_1) permettant d'établir et de vérifier les liens entre exigences de différents niveaux.
- SSS (System/Subsystem Specification) : document des exigences client.
- SRS (Software Requirements Specification) : document des exigences projet, qui référencent les exigences SSS.
- SDD (Software Design Document) : document de conception, qui référence les exigences SRS.
- Traçabilité : capacité à établir et vérifier les liens entre SSS, SRS et SDD. La chaîne va de SSS → SRS → SDD.
- Couverture : pourcentage d'exigences d'un niveau couvertes par le niveau suivant. Taux global = Taux SSS→SRS × Taux SRS→SDD.
- Lien brisé : référence vers un identifiant qui n'existe pas dans les documents analysés.
- Exigence orpheline : exigence projet (SRS) ne référençant aucune exigence client (SSS).
- Pattern / Regex : expression régulière configurable utilisée pour détecter automatiquement les identifiants d'exigences dans les documents.

4. Tests fonctionnels

Les tests fonctionnels vérifient le comportement de chaque module de l'outil de façon indépendante. Ils sont organisés en fiches de test, chacune correspondant à une fonctionnalité spécifique. Chaque fiche contient deux tableaux : le tableau des objectifs (ce qu'on vérifie et les pré-conditions) et le tableau des pas de tests numérotés.

4.1. Pour chaque scénario :

4.1.1. Identification

Chaque test est identifié par un code unique suivant la nomenclature définie en section 2.3.

4.1.2. Description

Chaque test décrit de façon concise l'objectif visé : quelle fonctionnalité est vérifiée, dans quelles conditions, et quel comportement est attendu en sortie.

4.1.3. Contraintes

Les pré-conditions constituent les hypothèses de départ du test : l'état du système et les fichiers disponibles avant d'exécuter l'action. Elles permettent d'isoler exactement ce qui est testé.

4.1.4. Dépendances

Certains tests dépendent de résultats d'autres tests. Ces dépendances sont indiquées dans les pré-conditions.

4.1.5. Procédure de test

La procédure décrit les pas de tests numérotés séquentiellement. Chaque pas décrit une action précise et le résultat attendu. La colonne Résultat indique le verdict obtenu lors de l'exécution.

Module 1 — Import & Parsing

Module 1 — Import & Parsing | Test fonctionnel nominal | EX-01, EX-05, EX-07

Fiche 01 — Import fichier SSS au format .docx Statut : PASS		
N° Objectif	Description — Ce qu'il y a à vérifier	Pré-condition(s)
OBJ_N_IMP_EX01_A	Vérifier que l'outil importe correctement un fichier SSS au format .docx, décompresse son archive ZIP interne, extrait le contenu XML et détecte exactement le nombre d'exigences présentes, sans erreur ni crash.	• L'application est compilée et lancée. • Le fichier sss_test.docx (5 exigences SSS valides) est disponible sur le disque. • Aucun fichier n'est déjà chargé dans l'interface.

N° Pas	Action	Résultat attendu	Résultat
OBJ_N_IMP_EX01_A — Import fichier SSS .docx valide			
1	Lancer l'application depuis le terminal ou l'IDE.	L'interface graphique s'ouvre sans message d'erreur.	
2	Localiser le bouton "Ouvrir" (ou équivalent) dans l'interface pour charger un fichier SSS et cliquer dessus.	Une fenêtre de sélection de fichier s'ouvre.	
3	Naviguer jusqu'au fichier sss_test.docx et le sélectionner.	Le fichier sss_test.docx est chargé. La fenêtre de sélection se ferme.	
4	Observer le compteur d'exigences affiché dans l'interface après le chargement.	Le compteur affiche exactement "5 exigences détectées". Aucune erreur. L'application ne plante pas.	PASS

Module 1 — Import & Parsing | Test fonctionnel nominal | EX-02, EX-07

Fiche 02 — Import fichier SRS au format .csv Statut : PASS

N° Objectif	Description — Ce qu'il y a à vérifier	Pré-condition(s)
OBJ_N_IMP_EX02_A	Vérifier que l'outil importe correctement un fichier SRS au format .csv, gère les séparateurs virgule et point-virgule, et détecte exactement le nombre d'exigences présentes.	- L'application est lancée. - Le fichier srs_test.csv (6 exigences SRS, séparateur point-virgule) est disponible. - Aucun fichier n'est déjà chargé.

N° Pas	Action	Résultat attendu	Résultat
OBJ_N_IMP_EX02_A — Import fichier SRS .csv valide			
1	Lancer l'application.	L'interface graphique s'ouvre sans erreur.	
2	Cliquer sur le bouton "Ouvrir" pour charger un fichier SRS.	La fenêtre de sélection s'ouvre.	
3	Sélectionner le fichier srs_test.csv.	Le fichier est chargé. La fenêtre se ferme.	
4	Observer le compteur d'exigences affiché après le chargement.	Le compteur affiche "6 exigences détectées". Le séparateur point-virgule est correctement géré. Aucune erreur.	PASS

Module 1 — Import & Parsing | Test fonctionnel nominal | EX-03, EX-07

Fiche 03 — Import fichier SDD au format .docx Statut : FAIL (.xls) PASS (.docx)

N° Objectif	Description — Ce qu'il y a à vérifier	Pré-condition(s)
OBJ_N_IMP_EX03_A	Vérifier que l'outil importe correctement un fichier SDD au format .docx et extrait les exigences de conception correctement.	- L'application est lancée. - Le fichier sdd_test.docx (5 exigences SDD valides) est disponible. - Aucun fichier n'est déjà chargé.

N° Pas	Action	Résultat attendu	Résultat
OBJ_N_IMP_EX03_A — Import fichier SDD .docx valide			
1	Lancer l'application.	L'interface graphique s'ouvre sans erreur.	
2	Cliquer sur le bouton "Ouvrir" pour charger un fichier SDD.	La fenêtre de sélection s'ouvre.	
3	Sélectionner le fichier sdd_test.docx.	Le fichier est chargé. La fenêtre se ferme.	
4	Observer le compteur d'exigences affiché après le chargement.	Le compteur affiche "5 exigences détectées". Aucun crash. Aucune erreur.	PASS

Module 1 — Import & Parsing | Test fonctionnel nominal | EX-04, EX-07

Fiche 04 — Import simultané SSS + SRS + SDD Statut : PASS

N° Objectif	Description — Ce qu'il y a à vérifier	Pré-condition(s)
OBJ_N_IMP_EX04_A	Vérifier que l'outil importe successivement les trois types de fichiers (SSS, SRS, SDD) et que le total des exigences correspond à la somme des trois fichiers, sans instabilité de l'application.	- L'application est lancée. - Les trois fichiers sss_test.docx (5 ex.), srs_test.csv (6 ex.) et sdd_test.docx (5 ex.) sont disponibles. - Aucun fichier n'est déjà chargé.

N° Pas	Action	Résultat attendu	Résultat
OBJ_N_IMP_EX04_A — Import multi-fichiers SSS + SRS + SDD			

N° Pas	Action	Résultat attendu	Résultat
1	Lancer l'application.	L'interface graphique s'ouvre sans erreur.	
2	Cliquer sur "Ouvrir" et sélectionner sss_test.docx.	Le fichier SSS est chargé. Compteur affiche 5 exigences SSS.	
3	Cliquer sur "Ouvrir" et sélectionner srs_test.csv.	Le fichier SRS est chargé. Compteur mis à jour.	
4	Cliquer sur "Ouvrir" et sélectionner sdd_test.docx.	Le fichier SDD est chargé. Compteur mis à jour.	
5	Observer le total des exigences affichées après les trois imports.	Le total affiché est 16 (5+6+5). L'application reste stable et réactive. Aucun crash.	PASS

Module 1 — Import & Parsing | Test de robustesse | EX-06, EX-07, EX-09

Fiche 05 — Robustesse : erreurs d'import Statut : PASS		
N° Objectif	Description — Ce qu'il y a à vérifier	Pré-condition(s)
OBJ_R_IMP_EX06_A	Vérifier que l'outil affiche un message d'erreur explicite et ne plante pas lorsqu'un fichier d'un format non supporté (.pdf) est sélectionné.	- L'application est lancée. - Un fichier test.pdf est disponible. - Aucun fichier n'est chargé.
OBJ_R_IMP_EX09_A	Vérifier que l'outil affiche un message d'erreur explicite et ne plante pas lorsqu'un fichier .docx corrompu (archive ZIP invalide) est sélectionné.	- L'application est lancée. - Le fichier corrompu.docx (ZIP invalide) est disponible. - Aucun fichier n'est chargé.
OBJ_R_IMP_EX09_B	Vérifier que l'outil gère sans crash un fichier .docx valide mais vide (sans aucune exigence) et affiche un compteur à 0.	- L'application est lancée. - Le fichier vide.docx (document Word vide) est disponible. - Aucun fichier n'est chargé.

N° Pas	Action	Résultat attendu	Résultat
OBJ_R_IMP_EX06_A — Format non supporté (.pdf)			
1	Lancer l'application.	L'interface s'ouvre.	
2	Cliquer sur "Ouvrir" dans l'interface.	La fenêtre de sélection s'ouvre.	
3	Sélectionner le fichier test.pdf.	Le fichier est transmis à l'application.	
4	Observer le comportement de l'application.	Un message d'erreur "Format non supporté" s'affiche. L'application ne plante pas et revient à l'état précédent.	PASS
OBJ_R_IMP_EX09_A — Fichier .docx corrompu			
5	Cliquer sur "Ouvrir" dans l'interface.	La fenêtre de sélection s'ouvre.	
6	Sélectionner le fichier corrompu.docx.	Le fichier est transmis à l'application.	
7	Observer le comportement lors de la tentative de décompression ZIP.	Un message d'erreur explicite s'affiche. L'application ne plante pas. Un nouvel import est possible.	PASS
OBJ_R_IMP_EX09_B — Fichier vide (0 exigence)			
8	Cliquer sur "Ouvrir" dans l'interface.	La fenêtre de sélection s'ouvre.	
9	Sélectionner le fichier vide.docx.	Le fichier est chargé.	
10	Observer le compteur d'exigences après le chargement.	Le compteur affiche "0 exigence détectée". Aucun crash. Un message informatif peut être affiché.	PASS

Module 1 — Import & Parsing | Test de robustesse | EX-10

Fiche 06 — Portabilité multi-OS Statut : —

N° Objectif	Description — Ce qu'il y a à vérifier	Pré-condition(s)
OBJ_N_IMP_EX10_A	Vérifier que l'outil s'installe et s'exécute correctement sur Windows, Linux et macOS sans comportement spécifique à un OS.	- Les installeurs pour Windows (.exe), Linux et macOS (.dmg) sont disponibles. - Le fichier sss_test.docx est accessible sur les trois machines.

N° Pas	Action	Résultat attendu	Résultat
OBJ_N_IMP_EX10_A — Portabilité Windows / Linux / macOS			
1	Sur Windows : double-cliquer sur l'installateur .exe et suivre les étapes d'installation.	L'installation se termine sans erreur sur Windows.	
2	Sur Windows : lancer l'outil installé et importer sss_test.docx.	L'outil s'exécute. Le fichier est importé. Compteur affiche 5 exigences.	
3	Sur Linux : lancer l'installateur et suivre les étapes d'installation.	L'installation se termine sans erreur sur Linux.	
4	Sur Linux : lancer l'outil installé et importer sss_test.docx.	L'outil s'exécute. Compteur affiche 5 exigences. Comportement identique à Windows.	
5	Sur macOS : ouvrir le fichier .dmg et suivre les étapes d'installation.	L'installation se termine sans erreur sur macOS.	
6	Sur macOS : lancer l'outil installé et importer sss_test.docx.	L'outil s'exécute. Compteur affiche 5 exigences. Comportement identique aux deux autres OS.	PASS

Module 1 — Parsing | Test fonctionnel nominal | EX-08, EX-11

Fiche 07 — Détection des identifiants par regex Statut : PASS

N° Objectif	Description — Ce qu'il y a à vérifier	Pré-condition(s)
OBJ_N_PAR_EX11_A	Vérifier que le moteur regex détecte exactement les identifiants valides dans un fichier contenant un mélange d'identifiants et de texte parasite, sans faux positif ni faux négatif.	- L'application est lancée. - Le fichier sss_test.docx est disponible avec 5 identifiants valides et du texte parasite ("Alloué à", "Non développé", "Test", "Le logiciel DOIT..."). - Le fichier a été importé avec succès (pas 1 à 4 de la Fiche 01 réalisés).
OBJ_N_PAR_EX11_B	Vérifier que pour chaque identifiant détecté, l'objet C++ ExigenceSSS est correctement construit en mémoire avec tous ses champs renseignés (id, contenu, typeDocument, fichierSource).	- Le fichier sss_test.docx a été importé avec succès. - Un affichage debug (std::cout) est disponible dans le code pour inspecter les objets créés.

N° Pas	Action	Résultat attendu	Résultat
OBJ_N_PAR_EX11_A — Détection regex sans faux positifs			
1	Importer le fichier sss_test.docx (voir Fiche 01, pas 1 à 4).	Le fichier est chargé. Compteur affiche 5 exigences.	
2	Cliquer sur "Lancer l'analyse" pour déclencher le parsing.	L'analyse démarre.	
3	Observer le nombre d'identifiants détectés par le moteur regex.	Exactement 5 identifiants sont détectés. Les chaînes "Alloué à", "Non développé", "Test", "Le logiciel DOIT..." ne sont pas comptées. 0 faux positif, 0 faux négatif.	PASS
OBJ_N_PAR_EX11_B — Construction des objets Exigence en mémoire			
4	Après l'import de sss_test.docx, vérifier en console debug les objets ExigenceSSS créés.	5 objets ExigenceSSS sont créés en mémoire.	

N° Pas	Action	Résultat attendu	Résultat
5	Pour chaque objet, vérifier que le champ 'id' est renseigné (ex : EXIGENCE_CLIENT_A_F01).	Le champ id est correctement renseigné pour chaque objet.	
6	Vérifier que le champ 'typeDocument' vaut SSS.	Le champ typeDocument vaut SSS pour tous les objets.	
7	Vérifier que le champ 'fichierSource' contient le nom du fichier importé.	Le champ fichierSource contient sss_test.docx.	PASS

Module 1 — Parsing | Test fonctionnel nominal | EX-08, EX-11, EX-12

Fiche 08 — Extraction du champ de traçabilité Statut : —		
N° Objectif	Description — Ce qu'il y a à vérifier	Pré-condition(s)
OBJ_N_PAR_EX12_A	Vérifier que lors du parsing d'un fichier SRS, l'outil extrait correctement le champ de traçabilité de chaque exigence et le stocke dans l'objet ExigenceSRS correspondant.	- L'application est lancée. - Le fichier srs_test.csv est disponible avec des exigences SRS dont le champ Traçabilité est renseigné (ex : EXIGENCE_PROJET_A_F01 → EXIGENCE_CLIENT_A_F01). - Un affichage debug est disponible pour inspecter les objets créés.

N° Pas	Action	Résultat attendu	Résultat
OBJ_N_PAR_EX12_A — Extraction du champ Traçabilité			
1	Lancer l'application.	L'interface s'ouvre.	
2	Importer le fichier srs_test.csv via le bouton "Ouvrir".	Le fichier est chargé. Compteur affiche 6 exigences SRS.	
3	Déclencher l'analyse via "Lancer l'analyse".	L'analyse démarre.	
4	Vérifier en console debug que l'objet EXIGENCE_PROJET_A_F01 contient un lien de traçabilité vers EXIGENCE_CLIENT_A_F01.	Le champ tracabilite de l'objet ExigenceSRS contient bien EXIGENCE_CLIENT_A_F01.	
5	Vérifier que l'objet EXIGENCE_PROJET_A_F06 a un champ traçabilité vide (orpheline).	Le champ tracabilite est vide pour EXIGENCE_PROJET_A_F06. L'objet est quand même créé sans crash.	PASS

Module 1 — Parsing | Test de robustesse | EX-08, EX-09

Fiche 09 — Robustesse du parsing Statut : PASS		
N° Objectif	Description — Ce qu'il y a à vérifier	Pré-condition(s)
OBJ_R_PAR_EX09_A	Vérifier que le programme ne plante pas et affiche 0 exigence lorsque le fichier importé ne contient aucun identifiant valide (texte narratif uniquement).	- L'application est lancée. - Un fichier .docx contenant uniquement du texte narratif sans identifiant en majuscules est disponible.
OBJ_R_PAR_EX09_B	Vérifier que le programme gère proprement les doublons d'identifiants : un identifiant présent deux fois ne doit être compté qu'une seule fois.	- L'application est lancée. - Un fichier .docx contenant deux occurrences du même identifiant EXIGENCE_CLIENT_A_F01 est disponible.

N° Pas	Action	Résultat attendu	Résultat
OBJ_R_PAR_EX09_A — Fichier sans identifiant valide			
1	Lancer l'application et importer un fichier .docx contenant uniquement du texte narratif.	Le fichier est chargé.	
2	Déclencher l'analyse.	L'analyse s'exécute.	

N° Pas	Action	Résultat attendu	Résultat
3	Observer le compteur d'exigences.	Le compteur affiche "0 exigence détectée". Le programme ne plante pas.	PASS
OBJ_R_PAR_EX09_B — Fichier avec identifiants en doublons			
4	Importer un fichier .docx contenant deux occurrences de EXIGENCE_CLIENT_A_F01.	Le fichier est chargé.	
5	Déclencher l'analyse.	L'analyse s'exécute.	
6	Observer le nombre d'identifiants détectés.	L'identifiant EXIGENCE_CLIENT_A_F01 est compté une seule fois. Aucun doublon. Le programme ne plante pas.	PASS

Module 2 — Moteur de Traçabilité

Module 2 — Moteur de Traçabilité | Test fonctionnel nominal | EX-12, EX-13, EX-16

Fiche 10 — Traçabilité SSS→SRS et exigences non couvertes Statut : PASS		
N° Objectif	Description — Ce qu'il y a à vérifier	Pré-condition(s)
OBJ_N_TRC_EX12_A	Vérifier que le moteur établit correctement les liens SSS→SRS et calcule le taux de couverture SSS→SRS lorsque les exigences SSS sont couvertes par des exigences SRS.	- Les fichiers sss_test.docx (5 SSS) et srs_test.csv (6 SRS dont F01 à F04 couvrent chacune une SSS) sont importés. - Le parsing a été réalisé sans erreur.
OBJ_N_TRC_EX13_A	Vérifier que le moteur détecte et signale correctement les exigences SSS non couvertes par aucune exigence SRS, et calcule le taux à 33%.	- Le fichier sss_test_trc13.docx (3 SSS : F01, F02, F03) est importé. - Le fichier srs_test_trc13.csv (1 SRS couvrant uniquement F01) est importé. - Le parsing a été réalisé sans erreur.

N° Pas	Action	Résultat attendu	Résultat
OBJ_N_TRC_EX12_A — Taux SSS→SRS			
1	Importer sss_test.docx puis srs_test.csv.	5 SSS et 6 SRS chargées.	
2	Cliquer sur "Lancer l'analyse".	L'analyse démarre dans le thread d'arrière-plan.	
3	Attendre la fin de l'analyse et observer le taux SSS→SRS affiché.	Le taux SSS→SRS affiché est 80% (4 SSS couvertes sur 5, F05 ayant un lien brisé). Les liens sont visibles dans la matrice.	PASS
OBJ_N_TRC_EX13_A — Détection exigences SSS non couvertes			
4	Importer sss_test_trc13.docx puis srs_test_trc13.csv.	3 SSS et 1 SRS chargées.	
5	Cliquer sur "Lancer l'analyse".	L'analyse démarre.	
6	Observer le taux SSS→SRS affiché.	Le taux SSS→SRS affiché est 33% (1 couverte sur 3).	
7	Observer la liste des anomalies pour identifier les exigences non couvertes.	EXIGENCE_CLIENT_A_F02 et EXIGENCE_CLIENT_A_F03 sont signalées comme "non couvertes". Le programme ne plante pas.	PASS

Module 2 — Moteur de Traçabilité | Test fonctionnel nominal | EX-12, EX-13, EX-16

Fiche 11 — Traçabilité SRS→SDD Statut : PASS		
N° Objectif	Description — Ce qu'il y a à vérifier	Pré-condition(s)

OBJ_N_TRC_EX12_B	Vérifier que le moteur établit correctement les liens SRS→SDD, identifie les exigences SRS non couvertes par un SDD, et calcule le taux SRS→SDD à 83.3%.	• Les fichiers srs_test.csv (6 SRS) et sdd_test.docx (5 SDD couvrant F01 à F05) sont importés. • Le parsing a été réalisé sans erreur.
-------------------------	--	---

N° Pas	Action	Résultat attendu	Résultat
OBJ_N_TRC_EX12_B — Taux SRS→SDD partiel			
1	Importer srs_test.csv puis sdd_test.docx.	6 SRS et 5 SDD chargées.	
2	Cliquer sur "Lancer l'analyse".	L'analyse démarre.	
3	Observer le taux SRS→SDD affiché.	Le taux SRS→SDD affiché est 83.3% (5 couvertes sur 6).	
4	Observer la liste des anomalies pour identifier l'exigence SRS non couverte.	EXIGENCE_PROJET_A_F06 est signalée comme non couverte par aucun SDD. Le programme ne plante pas.	PASS

Module 2 — Moteur de Traçabilité | Test fonctionnel nominal | EX-16

Fiche 12 — Calcul du taux de couverture global Statut : PASS		
N° Objectif	Description — Ce qu'il y a à vérifier	Pré-condition(s)
OBJ_N_TRC_EX16_A	Vérifier que le moteur calcule correctement le taux global selon la formule du CdC : Taux global = Taux SSS→SRS × Taux SRS→SDD, et que le résultat est cohérent avec les taux intermédiaires affichés.	• Les fichiers sss_test.docx (5 SSS), srs_test.csv (6 SRS) et sdd_test.docx (5 SDD) sont importés. • Le parsing a été réalisé sans erreur.

N° Pas	Action	Résultat attendu	Résultat
OBJ_N_TRC_EX16_A — Taux global = Taux SSS→SRS × Taux SRS→SDD			
1	Importer sss_test.docx, srs_test.csv et sdd_test.docx.	Les trois fichiers sont chargés : 5 SSS, 6 SRS, 5 SDD.	
2	Cliquer sur "Lancer l'analyse".	L'analyse démarre.	
3	Relever le taux SSS→SRS affiché.	Taux SSS→SRS = 80% (4 couvertes sur 5).	
4	Relever le taux SRS→SDD affiché.	Taux SRS→SDD = 83.3% (5 couvertes sur 6).	
5	Relever le taux global affiché et vérifier qu'il correspond à 80% × 83.3%.	Taux global affiché = 66.7%. Le résultat est cohérent avec la formule du CdC. Aucune erreur.	PASS

Module 2 — Moteur de Traçabilité | Test fonctionnel nominal | EX-14, EX-15

Fiche 13 — Détection des anomalies : liens brisés et orphelines Statut : PASS		
N° Objectif	Description — Ce qu'il y a à vérifier	Pré-condition(s)
OBJ_N_TRC_EX15_A	Vérifier que le moteur détecte et signale un lien brisé lorsqu'une exigence SRS référence un identifiant SSS qui n'existe pas dans le fichier SSS importé.	• Le fichier sss_test.docx (5 SSS : F01 à F05) est importé. • Le fichier srs_test.csv est importé : EXIGENCE_PROJET_A_F05 pointe vers EXIGENCE_CLIENT_A_F99 (inexistant dans le SSS).
OBJ_N_TRC_EX14_A	Vérifier que le moteur détecte et signale une exigence SRS orpheline lorsqu'une exigence SRS n'a aucun lien de traçabilité renseigné.	• Le fichier srs_test.csv est importé : EXIGENCE_PROJET_A_F06 a un champ Traçabilité vide.

N° Pas	Action	Résultat attendu	Résultat
OBJ_N_TRC_EX15_A — Détection lien brisé			
1	Importer sss_test.docx puis srs_test.csv.	5 SSS et 6 SRS chargées.	
2	Cliquer sur "Lancer l'analyse".	L'analyse démarre.	
3	Observer la liste des anomalies dans l'interface.	Le lien EXIGENCE_PROJET_A_F05 → EXIGENCE_CLIENT_A_F99 est signalé comme "lien brisé". L'analyse se poursuit sans crash.	PASS
OBJ_N_TRC_EX14_A — Détection exigence orpheline			
4	Observer la liste des anomalies après l'analyse.	EXIGENCE_PROJET_A_F06 est signalée comme "orpheline" (exigence SRS sans référence à une exigence SSS). Elle apparaît dans les anomalies.	PASS

Module 2 — Moteur de Traçabilité | Test fonctionnel nominal | EX-17

Fiche 14 — Génération du rapport exportable Statut : PASS		
N° Objectif	Description — Ce qu'il y a à vérifier	Pré-condition(s)
OBJ_N_TRC_EX17_A	Vérifier que l'outil génère un fichier rapport exportable contenant la date d'analyse, les trois taux de couverture, et la liste complète des anomalies (non couvertes, orphelines, liens brisés).	- Les fichiers sss_test.docx, srs_test.csv et sdd_test.docx sont importés. - L'analyse a été lancée et s'est terminée avec succès. - Le rapport affiché contient des taux et des anomalies.

N° Pas	Action	Résultat attendu	Résultat
OBJ_N_TRC_EX17_A — Export du rapport d'analyse			
1	Importer sss_test.docx, srs_test.csv et sdd_test.docx puis lancer l'analyse.	L'analyse se termine. Les taux et anomalies sont affichés.	
2	Localiser et cliquer sur le bouton "Exporter le rapport" (ou équivalent) dans l'interface.	L'outil déclenche la génération du rapport.	
3	Vérifier que le fichier rapport est bien créé sur le disque.	Un fichier rapport est généré (ex : rapport_tracabilite.txt ou .csv).	
4	Ouvrir le fichier rapport et vérifier qu'il contient la date de l'analyse.	La date et heure de l'analyse sont présentes dans le rapport.	
5	Vérifier que le rapport contient les trois taux (SSS→SRS, SRS→SDD, global).	Les trois taux sont présents et correspondent aux valeurs affichées dans l'interface.	
6	Vérifier que le rapport liste les anomalies : lien brisé et orpheline.	Les anomalies EXIGENCE_PROJET_A_F05 (lien brisé) et EXIGENCE_PROJET_A_F06 (orpheline) sont présentes dans le rapport.	PASS

Module 2 — Moteur de Traçabilité | Test de robustesse | EX-08, EX-09, EX-16, EX-22

Fiche 15 — Robustesse du moteur de traçabilité Statut : PASS		
N° Objectif	Description — Ce qu'il y a à vérifier	Pré-condition(s)
OBJ_R_TRC_EX16_A	Vérifier qu'il n'y a pas de division par zéro et que l'application ne plante pas lorsqu'un fichier SSS vide (0 exigence) est analysé.	- Le fichier vide.docx (0 exigence SSS) est importé. - Le fichier srs_test.csv est importé. - Le parsing a été réalisé.
OBJ_R_TRC_EX08_A	Vérifier que l'analyse de 100 exigences (50 SSS + 30 SRS + 20 SDD) se termine en moins de 5 secondes sans blocage visible de l'interface.	Les fichiers sss_perf.docx (50 ex.), srs_perf.csv (30 ex.) et sdd_perf.docx (20 ex.) sont disponibles.

		• Un chronomètre est disponible pour mesurer le temps.
--	--	--

N° Pas	Action	Résultat attendu	Résultat
OBJ_R_TRC_EX16_A — Aucune division par zéro avec SSS vide			
1	Importer le fichier vide.docx comme fichier SSS.	Le fichier est chargé. Compteur affiche 0 exigence SSS.	
2	Importer srs_test.csv.	6 exigences SRS chargées.	
3	Cliquer sur "Lancer l'analyse".	L'analyse s'exécute.	
4	Observer le comportement de l'application et les taux affichés.	Aucune division par zéro. Le taux est affiché comme 0% ou N/A. L'application ne plante pas.	PASS
OBJ_R_TRC_EX08_A — Performance : 100 exigences en < 5 secondes			
5	Importer sss_perf.docx (50 SSS), srs_perf.csv (30 SRS) et sdd_perf.docx (20 SDD).	Les trois fichiers sont chargés. Total 100 exigences.	
6	Déclencher le chronomètre et cliquer sur "Lancer l'analyse".	L'analyse démarre.	
7	Arrêter le chronomètre à l'affichage du message de fin d'analyse.	L'analyse se termine en moins de 5 secondes. L'interface reste réactive pendant l'exécution.	PASS

Module 3 — Interface Graphique

Module 3 — Interface Graphique | Test fonctionnel nominal | EX-18, EX-19

Fiche 16 — Affichage matrice et code couleur Statut : PASS		
N° Objectif	Description — Ce qu'il y a à vérifier	Pré-condition(s)
OBJ_N_GUI_EX18_A	Vérifier que la matrice de traçabilité s'affiche correctement après analyse, avec les exigences SSS en lignes et les exigences SRS/SDD en colonnes, et que chaque case indique si le lien existe.	- Les fichiers sss_test.docx, srs_test.csv et sdd_test.docx sont importés. - L'analyse a été lancée et s'est terminée avec succès.
OBJ_N_GUI_EX19_A	Vérifier que le code couleur est correctement appliqué dans la matrice : vert pour les exigences couvertes, rouge pour les non couvertes, orange pour les liens brisés.	- Les fichiers sss_test.docx, srs_test.csv et sdd_test.docx sont importés. - L'analyse a été lancée. Le rapport contient les 3 statuts : couvert, non couvert, lien brisé.

N° Pas	Action	Résultat attendu	Résultat
OBJ_N_GUI_EX18_A — Affichage de la matrice			
1	Importer sss_test.docx, srs_test.csv et sdd_test.docx puis lancer l'analyse.	L'analyse se termine. Les résultats sont disponibles.	
2	Observer la zone matrice dans l'interface graphique.	La matrice s'affiche avec les exigences SSS en lignes et SRS/SDD en colonnes.	
3	Vérifier qu'une case indique la présence ou l'absence d'un lien pour chaque couple SSS/SRS.	Chaque case de la matrice indique correctement si le lien existe ou non.	PASS
OBJ_N_GUI_EX19_A — Code couleur vert / rouge / orange			
4	Observer la couleur des cases correspondant aux exigences couvertes.	Les cases couvertes sont affichées en vert.	
5	Observer la couleur de la case correspondant à l'exigence SSS non couverte (F05 dont le SRS a un lien brisé).	La case est affichée en rouge (non couverte).	

N° Pas	Action	Résultat attendu	Résultat
6	Observer la couleur des cases correspondant au lien brisé (EXIGENCE_PROJET_A_F05 → EXIGENCE_CLIENT_A_F99 inexistant).	La case du lien brisé est affichée en orange. Les couleurs correspondent au code défini dans le CdC.	PASS

Module 3 — Interface Graphique | Test fonctionnel nominal | EX-20, EX-21

Fiche 17 — Affichage des taux et filtrage Statut : PASS			
N° Objectif	Description — Ce qu'il y a à vérifier	Pré-condition(s)	
OBJ_N_GUI_EX21_A	Vérifier que les trois taux de couverture (SSS→SRS, SRS→SDD, global) sont affichés dans l'interface et correspondent exactement aux valeurs calculées par le moteur.	- Les fichiers sss_test.docx, srs_test.csv et sdd_test.docx sont importés. - L'analyse a été lancée et s'est terminée.	
OBJ_N_GUI_EX20_A	Vérifier que le filtre par statut fonctionne correctement : en sélectionnant "Non couvert", seules les exigences non couvertes doivent être affichées dans la matrice.	- L'analyse a été lancée et s'est terminée. - La matrice affiche des exigences de plusieurs statuts (couvertes, non couvertes, liens brisés).	

N° Pas	Action	Résultat attendu	Résultat
OBJ_N_GUI_EX21_A — Affichage des taux			
1	Importer les trois fichiers et lancer l'analyse.	L'analyse se termine.	
2	Localiser la zone d'affichage des taux dans l'interface.	Trois taux sont visibles : SSS→SRS, SRS→SDD, et global.	
3	Vérifier que le taux SSS→SRS affiché est 80%.	Le taux SSS→SRS affiché est bien 80%.	
4	Vérifier que le taux SRS→SDD affiché est 83.3%.	Le taux SRS→SDD affiché est bien 83.3%.	
5	Vérifier que le taux global affiché est 66.7%.	Le taux global affiché est 66.7%. Les valeurs correspondent aux calculs du moteur.	PASS
OBJ_N_GUI_EX20_A — Filtrage par statut 'Non couvert'			
6	Localiser le filtre par statut dans l'interface et sélectionner "Non couvert".	Le filtre est appliqué.	
7	Observer la matrice après application du filtre.	Seules les exigences non couvertes sont affichées. Les exigences couvertes disparaissent. Aucun crash.	
8	Désactiver le filtre et observer que la matrice revient à son état complet.	La matrice affiche à nouveau toutes les exigences.	PASS

Module 3 — Interface Graphique | Test de robustesse | EX-09, EX-22

Fiche 18 — Robustesse de l'interface graphique Statut : PASS			
N° Objectif	Description — Ce qu'il y a à vérifier	Pré-condition(s)	
OBJ_R_GUI_EX09_A	Vérifier que le bouton "Lancer l'analyse" est désactivé ou qu'un message d'erreur s'affiche si aucun fichier n'a été importé, empêchant un crash dû à des données vides.	- L'application vient d'être lancée. - Aucun fichier n'a été importé.	
OBJ_R_GUI_EX22_A	Vérifier que l'outil fonctionne intégralement en local sans aucune connexion réseau, conformément à l'exigence EX-22.	- L'application est lancée. - La connexion Wi-Fi est coupée sur la machine de test. - Les fichiers de test sont disponibles localement.	

N° Pas	Action	Résultat attendu	Résultat
OBJ_R_GUI_EX09_A — Analyse sans fichier importé			
1	Lancer l'application sans importer aucun fichier.	L'interface s'ouvre.	
2	Observer l'état du bouton "Lancer l'analyse".	Le bouton est désactivé (grisé) OU un message s'affiche : "Veuillez d'abord importer les fichiers". L'application ne plante pas.	N/A — bouton désactivé tant que SSS et SRS ne sont pas chargés
OBJ_R_GUI_EX22_A — Fonctionnement sans réseau			
3	Couper la connexion Wi-Fi sur la machine de test.	La machine n'a plus accès à internet.	
4	Importer sss_test.docx, srs_test.csv et sdd_test.docx.	Les fichiers sont chargés sans erreur réseau.	
5	Lancer l'analyse et observer les résultats.	L'analyse se termine. Les taux et anomalies s'affichent. Aucune erreur réseau.	
6	Réactiver la connexion Wi-Fi.	La connexion est rétablie. Le comportement de l'outil n'a pas changé.	PASS

5. Tests d'intégration

Les tests d'intégration vérifient la communication entre les modules : M1→M2 (transmission des objets d'exigences) et M2→M3 (transmission du rapport à la GUI). Ils s'assurent qu'aucune donnée n'est perdue ou corrompue lors des échanges inter-modules.

5.1. Pour chaque test d'intégration :

5.1.1. Identification

Les tests d'intégration suivent la même nomenclature que les tests fonctionnels, avec le module INT pour les distinguer.

5.1.2. Description

Chaque test décrit quel flux de données entre deux modules est vérifié : quantité, intégrité des identifiants, fidélité du rapport affiché.

5.1.3. Contraintes

Les tests d'intégration supposent que les tests fonctionnels des modules concernés sont déjà validés (PASS).

5.1.4. Dépendances

Les tests d'intégration M2→M3 dépendent de la validation préalable des tests M1→M2.

5.1.5. Procédure de test

La vérification se fait soit par observation de la GUI, soit par affichage debug dans la console C++ (std::cout sur le nombre d'objets reçus par le module).

Tests d'intégration | EX-08, EX-09, EX-11, EX-12

Fiche 19 — Intégration Module 1 → Module 2 Statut : N/A		
N° Objectif	Description — Ce qu'il y a à vérifier	Pré-condition(s)
OBJ_N_INT_EX11_A	Vérifier que le Module 2 reçoit exactement le même nombre d'exigences que celles extraites par le Module 1, sans perte, duplication ni modification lors de la transmission.	- Le fichier sss_test.docx (5 exigences SSS) a été importé et parsé par le Module 1. - Un affichage debug est disponible pour vérifier les données reçues par le Module 2.
OBJ_N_INT_EX12_A	Vérifier que les identifiants des exigences sont transmis sans modification entre le Module 1 et le Module 2 (pas de changement de casse, d'espaces ou de caractères).	- Le fichier sss_test.docx a été importé. Il contient l'identifiant EXIGENCE_CLIENT_A_F01. - Un affichage debug est disponible.

N° Pas	Action	Résultat attendu	Résultat
OBJ_N_INT_EX11_A — Transmission complète M1→M2			
1	Importer sss_test.docx. Observer le compteur affiché par le Module 1.	Module 1 affiche : 5 exigences SSS extraites.	
2	Cliquer sur "Lancer l'analyse". Observer via debug le nombre d'objets reçus par le Module 2.	Le Module 2 reçoit exactement 5 objets ExigenceSSS. Aucune exigence perdue ou dupliquée.	
3	Vérifier que le taux calculé par le moteur est basé sur 5 exigences SSS.	Le taux SSS→SRS est calculé sur une base de 5 exigences, confirmant la bonne transmission.	N/A — couvert par OBJ_N_IMP_EX01_A
OBJ_N_INT_EX12_A — Intégrité des identifiants M1→M2			
4	Importer sss_test.docx et déclencher l'analyse.	L'analyse démarre.	
5	Via debug, vérifier l'identifiant de la première exigence reçue par le moteur.	L'identifiant reçu est exactement EXIGENCE_CLIENT_A_F01 : casse,	N/A — couvert par OBJ_N_PAR_EX12_A

N° Pas	Action	Résultat attendu	Résultat
		underscores et chiffres identiques à la source. Aucune modification.	

Tests d'intégration | EX-08, EX-16, EX-18, EX-21

Fiche 20 — Intégration Module 2 → Module 3 Statut : N/A			
N° Objectif	Description — Ce qu'il y a à vérifier	Pré-condition(s)	
OBJ_N_INT_EX21_A	Vérifier que l'interface graphique (Module 3) affiche exactement les mêmes valeurs que celles calculées par le moteur (Module 2) : mêmes taux, mêmes anomalies, aucune donnée manquante.	- Les fichiers sss_test.docx, srs_test.csv et sdd_test.docx sont importés. - L'analyse a été lancée et s'est terminée.	
OBJ_N_INT_EX08_A	Vérifier que l'interface graphique reste réactive pendant l'exécution de l'analyse, prouvant que le ThreadManager fonctionne correctement.	- Les fichiers de performance sss_perf.docx (50 ex.), srs_perf.csv (30 ex.) et sdd_perf.docx (20 ex.) sont importés. - L'analyse n'a pas encore été lancée.	

N° Pas	Action	Résultat attendu	Résultat
OBJ_N_INT_EX21_A — Fidélité rapport M2→M3			
1	Importer les trois fichiers et lancer l'analyse.	L'analyse se termine. Les valeurs s'affichent dans l'interface.	
2	Comparer les taux affichés avec les valeurs attendues (80%, 83.3%, 66.7%).	Les taux affichés correspondent exactement aux valeurs calculées par le moteur. Aucune donnée manquante.	
3	Vérifier que le lien brisé et l'orpheline apparaissent dans l'interface.	Les deux anomalies sont présentes et correctement identifiées dans l'interface.	N/A — couvert par OBJ_N_GUI_EX18_A, EX19_A, EX21_A
OBJ_N_INT_EX08_A — Interface réactive pendant l'analyse (ThreadManager)			
4	Importer les fichiers de performance (100 exigences) et cliquer sur "Lancer l'analyse".	L'analyse démarre dans le thread d'arrière-plan.	
5	Pendant l'analyse, tenter d'interagir avec l'interface.	N/A — l'analyse des 100 exigences se termine trop rapidement pour permettre une interaction manuelle. Le bon fonctionnement du ThreadManager est vérifié par revue du code ThreadManager.cpp.	N/A — analyse trop rapide. Vérifié par revue du ThreadManager.cpp

6. Tests unitaires

Les tests unitaires vérifient le comportement de fonctions ou méthodes C++ de façon isolée, indépendamment des autres modules.

6.1. Pour chaque test unitaire :

6.1.1. Identification

Les tests unitaires sont identifiés par un code de la forme UT_[CLASSE]_[METHODE]_[N°].

6.1.2. Description

Chaque test unitaire vérifie le comportement d'une méthode spécifique pour un jeu d'entrées donné.

6.1.3. Contraintes

Les tests unitaires s'exécutent sans dépendances externes : pas de fichiers réels, pas d'interface graphique. Les données sont injectées directement dans le code.

6.1.4. Dépendances

Aucune dépendance inter-modules. Chaque test unitaire est indépendant.

6.1.5. Procédure de test

Les tests unitaires sont réalisés directement dans le main.cpp de test, ou via des assertions C++ (assert).

Les principales méthodes testées sont :

- MoteurTracabilite::analyserTracabilite() : vérification des taux calculés sur des données fictives injectées directement (voir Main.cpp du projet).
- ThreadManager::lancerAnalyse() / attendreFinAnalyse() : vérification que le thread se lance et se termine correctement.
- ExtracteurFichier : vérification de l'extraction du texte XML sur un fichier .docx de test minimal.

Note : à ce stade du projet, les tests unitaires sont réalisés manuellement via le Main.cpp fourni dans le code source, qui injecte des données fictives directement dans le moteur et vérifie les résultats affichés en console.

N° / Identifiant	Résumé	Pré-conditions	Action(s)	Résultat attendu	E X	Résultat
UT_MOTEUR_01	MoteurTracabilite::analyserTracabilite() : taux corrects sur données fictives	• Données fictives injectées directement dans le moteur (sans fichiers).	Appeler analyserTracabilite() avec 2 SSS, 2 SRS (couvrant les 2 SSS), 1 SDD (couvrant SRS-1).	tauxSSS_SRS=100%, tauxSRS_SDD=50%, tauxGlobal=50%.	—	PASS
UT_THREAD_01	ThreadManager::lancerAnalyse() / attendreFinAnalyse() : thread correct	• Moteur initialisé avec données fictives.	Lancer le thread. Attendre la fin. Vérifier le résultat.	Thread se lance et se termine correctement. Rapport disponible après.	—	PASS
UT_EXTRAC_T_01	ExtracteurFichier : extraction XML sur fichier .docx minimal	• Fichier sss_test.docx disponible.	Appeler extraire() sur le fichier de test.	Identifiants correctement extraits. Pas de crash.	—	PASS
UT_CONV_01	ConvertisseurExigences : détection de type depuis identifiant	• Identifiants tests : EXIGENCE_CLIENT_A_F01, EXIGENCE_PROJET_A_01, SDD_FUNC_01.	Appeler detecterType() sur chacun.	SSS, SRS, SDD détectés correctement.	—	PASS
UT_CONV_02	ConvertisseurExigences : transfert correct de la traçabilité	• ExigenceExtraite avec tracabilite = [EXIGENCE_CLIENT_A_F01].	Appeler convertir(). Vérifier	getTracabilite() retourne bien [EXIGENCE_CLIENT_A_F01].	—	PASS

N° / Identifiant	Résumé	Pré-conditions	Action(s)	Résultat attendu	E X	Résul tat
			getTracabilite() sur l'objet SRS.			

7. Vérification de couverture des fichiers de test

Cette section présente le détail du calcul de couverture pour les jeux de fichiers de test utilisés lors de la validation.

7.1. Jeu de test simple (sss_test.docx + srs_test.csv + sdd_test.docx)

Niveau	Total	Couvertes	Non couvertes	Taux
SSS (sss_test.docx)	5	4	1 (F05 — lien brisé vers F99)	80%
SRS (srs_test.csv)	6	5	1 (F06 — orpheline)	83.3%
Global	—	—	—	$80\% \times 83.3\% = 66.7\%$

7.2. Jeu de test multi-domaines (navigation + sécurité + paiement)

Niveau / Domaine	Total	Couvertes	Non couvertes	Taux
SSS Navigation	8	7	1 (NAV_F08)	7/8
SSS Sécurité	7	7	0	7/7
SSS Paiement	9	8	1 (PAY_F09)	8/9
TOTAL SSS→SRS	24	22	2	$22/24 = 91.7\%$
SRS Navigation	9	8	1 (NAV_09)	8/9
SRS Sécurité	7	7	0	7/7
SRS Paiement	8	8	0	8/8
TOTAL SRS→SDD	24	23	1	$23/24 = 95.8\%$
GLOBAL	—	—	—	$91.7\% \times 95.8\% = 87.8\%$

8. Vérification de la documentation

Cette section vérifie que l'ensemble des livrables documentaires attendus sont bien fournis et conformes.

Document	Vérification	Résultat
Code source C++	Code présent, compilable, commenté	PASS
Cahier des Charges (D5)	Document présent, version A01	PASS
Cahier de Recettes	Document présent avec scénarios	PASS
Cahier de Conception Générale	Document présent, version A01	PASS
Cahier de Conception Détaillée	Document présent	PASS
Cahier du Plan de Test (ce document)	Document présent	PASS
Fichier README	Instructions de compilation et d'utilisation présentes	—

9. Annexes

Fichiers de test utilisés lors de l'exécution du plan de test :

- sss_test.docx — 5 exigences SSS valides (cas nominal import et traçabilité)
- srs_test.csv — 6 exigences SRS (dont 1 lien brisé et 1 orpheline)
- sdd_test.docx — 5 exigences SDD valides
- sss_test_trc13.docx — 3 exigences SSS (test détection non couvertes)
- srs_test_trc13.csv — 1 exigence SRS (test 33% de couverture)
- sss_perf.docx — 50 exigences SSS (test performance)
- srs_perf.csv — 30 exigences SRS (test performance)
- sdd_perf.docx — 20 exigences SDD (test performance)
- corrompu.docx — archive ZIP invalide (test robustesse import)
- vide.docx — fichier sans aucune exigence (test robustesse parsing)

10. Glossaire

Terme	Définition
SSS	System/Subsystem Specification — document des exigences client.
SRS	Software Requirements Specification — document des exigences projet.
SDD	Software Design Document — document de conception.
Traçabilité	Capacité à relier les exigences SSS aux SRS et aux SDD.
Lien brisé	Référence vers un identifiant d'exigence inexistant dans les documents.
Exigence orpheline	Exigence SRS ne référençant aucune exigence SSS.
Pattern / Regex	Expression régulière utilisée pour détecter les identifiants.
Taux de couverture	Pourcentage d'exigences couvertes. Taux global = Taux SSS→SRS × Taux SRS→SDD.
PASS	Le comportement observé correspond au résultat attendu.
FAIL	Le comportement observé ne correspond pas au résultat attendu.
N/A	Non applicable — le test ne peut pas être réalisé dans le contexte du programme.
GUI	Graphical User Interface — interface graphique de l'outil.

11. Références

- Cahier des Charges du Projet L2G1 — version A02, Mai 2026
- Cahier des Recettes — version A02, Mai 2026
- Cahier de Conception Générale — référence D7, version A02, Mai 2026
- Cahier de Conception Détaillée — version A02, Mai 2026
- Documentation C++ standard (STL, threads, regex)
- Documentation bibliothèques : zlib/minizip, tinypug/pugixml

12. Index

Exigences du CdC couvertes par ce plan de test :

EX	Description	Test(s) associé(s)
EX-01	Import .docx	OBJ_N_IMP_EX01_A
EX-02	Import .csv	OBJ_N_IMP_EX02_A
EX-03	Import .xls / .docx SDD	OBJ_N_IMP_EX03_A
EX-04	Import multi-fichiers	OBJ_N_IMP_EX04_A
EX-05	Décompression ZIP .docx	OBJ_N_IMP_EX01_A
EX-06	Message erreur format non supporté	OBJ_R_IMP_EX06_A, OBJ_R_IMP_EX09_A
EX-07	Compteur d'exigences détectées	OBJ_N_IMP_EX01_A à EX04_A
EX-08	Performance — pas de blocage	OBJ_R_TRC_EX08_A, OBJ_N_PAR_EX11_A
EX-09	Robustesse — pas de crash	OBJ_R_IMP_EX09_A/B, OBJ_R_PAR_EX09_A/B
EX-10	Portabilité multi-OS	OBJ_N_IMP_EX10_A
EX-11	Détection identifiants par pattern	OBJ_N_PAR_EX11_A, OBJ_N_PAR_EX11_B
EX-12	Liens de traçabilité SSS↔SRS	OBJ_N_TRC_EX12_A, OBJ_N_TRC_EX12_B
EX-13	Exigences SSS non couvertes	OBJ_N_TRC_EX13_A
EX-14	Exigences SRS orphelines	OBJ_N_TRC_EX14_A
EX-15	Liens brisés	OBJ_N_TRC_EX15_A
EX-16	Calcul taux de couverture	OBJ_N_TRC_EX16_A
EX-17	Rapport exportable	OBJ_N_TRC_EX17_A
EX-18	Matrice de traçabilité	OBJ_N_GUI_EX18_A
EX-19	Code couleur	OBJ_N_GUI_EX19_A
EX-20	Filtrage des résultats	OBJ_N_GUI_EX20_A
EX-21	Affichage taux de couverture	OBJ_N_GUI_EX21_A
EX-22	Fonctionnement local sans réseau	OBJ_R_GUI_EX22_A, OBJ_R_TRC_EX08_A

13. Suivi des modifications

13.1. Version A.01 :

ID	Titre	Commentaires
7201	Rédaction des plans de tests	Rédaction et mise en place du cahier de plan de test
7203	Tests fonctionnels	Vérification des tests fonctionnels
7204	Tests intégration	Vérification des tests d'intégration
7205	Tests unitaires	Vérification des tests unitaires